

Pregunta 2: Implementación del Método de Diferencias Finitas

para EDOs de Segundo Orden con Condiciones de Frontera

Ejercicios del 28 de Abril del 2026 — GNU Octave

1. Estrategia para implementar el método de diferencias finitas

El método de diferencias finitas transforma una ecuación diferencial ordinaria (EDO) lineal de segundo orden con condiciones de frontera en un sistema de ecuaciones lineales algebraicas. La EDO general considerada es:

$$y''(x) = p(x)y'(x) + q(x)y(x) + r(x), \quad x \in [a, b],$$

con condiciones de frontera $y(a) = y_0$ e $y(b) = y_n$.

La estrategia consiste en discretizar el dominio $[a, b]$ en n subintervalos de tamaño $h = (b-a)/n$, generando nodos $x_j = a + jh$ para $j = 0, 1, \dots, n$. Luego se aproximan las derivadas primera y segunda mediante diferencias finitas centradas de orden $O(h^2)$:

$$y'(x_j) \approx (y_{j+1} - y_{j-1}) / (2h), \quad y''(x_j) \approx (y_{j-1} - 2y_j + y_{j+1}) / h^2.$$

Al sustituir estas aproximaciones en la EDO y reorganizar términos, se obtiene una ecuación algebraica en cada nodo interior x_j ($j = 1, \dots, n-1$) que relaciona y_{j-1} , y_j e y_{j+1} . Dado que y_0 e y_n son conocidos por las condiciones de frontera, se tiene un sistema de $(n-1)$ ecuaciones con $(n-1)$ incógnitas cuya matriz de coeficientes es tridiagonal.

2. Construcción del sistema lineal tridiagonal

Al sustituir las diferencias finitas centradas en la EDO y multiplicar ambos lados por h^2 , la ecuación en el nodo x_j queda:

$$-(1 + (h/2)p_j) y_{j-1} + (2 + h^2 q_j) y_j - (1 - (h/2)p_j) y_{j+1} = -h^2 r_j,$$

donde $p_j = p(x_j)$, $q_j = q(x_j)$ y $r_j = r(x_j)$. Esto define un sistema tridiagonal $A \cdot Y = d$, donde la matriz A tiene:

- Diagonal principal: $a_j = 2 + h^2 q_j$
- Diagonal superior: $c_j = -(1 - (h/2) p_j)$
- Diagonal inferior: $b_j = -(1 + (h/2) p_{j+1})$

El vector del lado derecho d tiene componentes $d_j = -h^2 r_j$, excepto d_1 que se le suma $(1 + (h/2)p_1) \cdot y_0$ y d_{n-1} que se le suma $(1 - (h/2)p_{n-1}) \cdot y_n$, para incorporar las condiciones de frontera conocidas.

3. Gráfica comparativa: aproximaciones vs. solución exacta

La siguiente gráfica muestra las aproximaciones numéricas obtenidas para cada valor de h , junto con la solución exacta $y(x) = \sin(6-x) / (\sin(5)\sqrt{x})$. Se observa cómo al reducir h , la aproximación converge hacia la solución exacta.

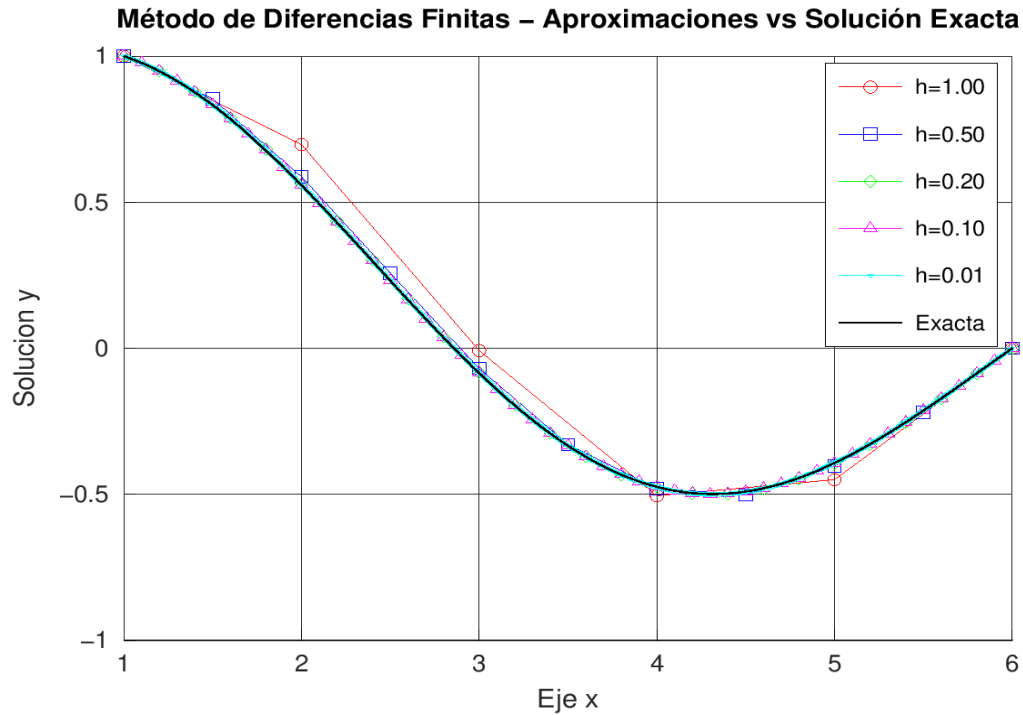


Figura 1: Aproximaciones por diferencias finitas para distintos valores de h y la solución exacta.

4. Tabla de errores absolutos máximos

La tabla siguiente resume el error absoluto máximo obtenido para cada tamaño de paso h . El error se calcula como $\max |y_{\text{aprox}}(x_j) - y_{\text{exacta}}(x_j)|$ sobre todos los nodos.

h	n (subintervalos)	Error absoluto maximo
1.00	5	1.3890e-01
0.50	10	2.9202e-02
0.20	25	4.4948e-03
0.10	50	1.1202e-03
0.01	500	1.1192e-05

Tabla 1: Error absoluto máximo para cada valor de h .

5. Comandos y funciones de GNU Octave utilizados

La implementación utiliza las siguientes funciones y comandos de Octave:

- **diag(v, k)**: Crea una matriz diagonal a partir del vector v. Con k=0 genera la diagonal principal, k=1 la superdiagonal y k=-1 la subdiagonal. Se utilizó para construir la matriz tridiagonal A.
- **mldivide(A, d)** (operador \): Resuelve el sistema lineal $Ax = d$. Para matrices tridiagonales, Octave aplica internamente un algoritmo eficiente equivalente al método de Thomas.
- **zeros(m, n)**: Crea una matriz de ceros de tamaño $m \times n$. Se utilizó para preasignar vectores y la matriz del sistema.
- **round(x)**: Redondea al entero más cercano. Se usó para calcular $n = (b-a)/h$ evitando errores de punto flotante.
- **max(v)** y **abs(v)**: Se combinan para calcular el error absoluto máximo entre la aproximación y la solución exacta.
- **plot, xlabel, ylabel, title, legend, grid**: Funciones de graficación utilizadas para generar la figura comparativa con todas las curvas de aproximación y la solución exacta.
- **print('-dpng', ...)**: Exporta la figura como imagen PNG para incluirla en el reporte.
- **linspace(a, b, n)**: Genera n puntos equiespaciados entre a y b. Se usó para graficar la solución exacta con alta resolución.
- **Funciones anónimas (@(x) ...)**: Se utilizaron para definir $p(x)$, $q(x)$, $r(x)$ y la solución exacta como handles de función, lo que permite pasar las funciones como parámetros a `edo2`.

6. Análisis de precisión y costo computacional

El método de diferencias finitas centradas tiene un error de truncamiento local de orden $O(h^2)$. Esto se verifica experimentalmente al observar la tabla de errores: al reducir h a la mitad, el error se reduce aproximadamente por un factor de 4. Por ejemplo, al pasar de $h=1.0$ (error $1.39e-01$) a $h=0.5$ (error $2.92e-02$), el factor de reducción es aproximadamente 4.76, consistente con la convergencia cuadrática $O(h^2)$.

En cuanto al costo computacional, el sistema tridiagonal de tamaño $(n-1) \times (n-1)$ se resuelve con el algoritmo de Thomas en $O(n)$ operaciones, lo que hace que el método sea muy eficiente. Sin embargo, al disminuir h (aumentando n), el número de operaciones y el uso de memoria crecen linealmente. Para $h=0.01$ se resuelve un sistema de 499 ecuaciones, lo cual sigue siendo computacionalmente ligero. La construcción de la matriz tridiagonal y la evaluación de las funciones p, q y r en todos los nodos también es $O(n)$.

En la práctica, la función `mldivide` de Octave detecta automáticamente la estructura tridiagonal de la matriz A y aplica un solucionador optimizado, por lo que no es necesario implementar manualmente el algoritmo de Thomas. Esto simplifica la implementación sin sacrificar rendimiento.

En conclusión, el método es preciso y eficiente para este tipo de problemas. Con $h=0.01$ se alcanza un error máximo del orden de 10^{-5} , lo cual demuestra la convergencia del esquema y su utilidad para la resolución numérica de EDOs lineales de segundo orden con condiciones de frontera.